

Safe Communication-Free MAPF

Chiawen Liao
chiawenl@andrew.cmu.edu

Thomas Chan
yuhsinch@andrew.cmu.edu

Abstract—Multi-agent path finding (MAPF) is a fundamental problem in multi-robot coordination. Centralized optimal solvers degrade in dense, large-scale settings, while decentralized learning-based approaches typically rely on inter-agent communication to resolve conflicts—an assumption that breaks down in subterranean environments, adversarial settings, or areas with heavy RF interference. We propose a communication-free variant of decentralized MAPF and address it through two complementary methods. First, we fine-tune MAPF-GPT on unsafe states mined using a local CS-PIBT safety shield, converting reactive shielding into proactive policy improvement. Second, we introduce an uncertainty-aware planner that leverages fixed priority orderings and a shared policy to simulate higher-priority agents, construct a cumulative occupancy risk map, and select actions that balance policy preference against collision risk. Preliminary benchmarking confirms that purely learning-based methods struggle in dense settings, but our PIBT-based data collection pipeline raised individual success rates from 0.35 to 1.00 in a 48-agent setup.

I. INTRODUCTION & MOTIVATION

Multi-agent path finding (MAPF) is a fundamental problem in multi-robot coordination: given a set of agents operating in a shared environment, the goal is to find collision-free paths from their respective start locations to their assigned goals. This problem arises in a wide range of real-world applications, including automated warehouses, autonomous vehicle coordination, and drone fleet management, where dozens to thousands of robots must navigate efficiently without colliding.

Finding an optimal solution to MAPF is NP-hard, meaning computational complexity grows exponentially with the number of agents. Optimal solvers such as [12] perform well in sparse environments but degrade significantly in dense, large-scale settings. To remain practical, solvers must trade optimality for computational efficiency. Several sub-optimal methods — including [7], [9], [10], and [15] — have been proposed to address this, offering bounded-suboptimal or complete solutions within tractable time budgets. Nevertheless, these methods are centralized: all agent paths are computed jointly by a single planner, and even with significant engineering effort, centralized approaches inevitably slow down as the number of agents scales.

Decentralized approaches offer a compelling alternative. When each robot is equipped with onboard sensors and computation, it can maintain a local observation of its surroundings and independently plan its next action at each timestep. A number of learning-based methods, such as [1] and [8], follow this paradigm by taking local observations as input and outputting a single-step action for each robot. However, this introduces a fundamental coordination challenge: since

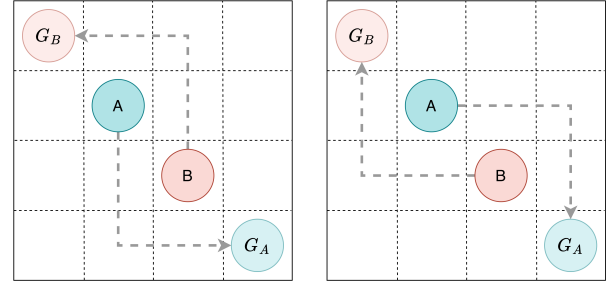


Fig. 1. Case may cause conflict

each robot plans independently without knowledge of others' intentions, resolving conflicts becomes difficult. Consider the configuration shown in Figure 1: two symmetric, equally optimal solutions exist, and two independently planning robots have no basis to prefer one over the other, making collisions likely. In safety-critical settings — such as a warehouse where one robot carries flammable materials — such collisions could have catastrophic consequences.

Recent works such as [13] and [14] partially mitigate this issue by using a learning-based planner as a first stage, then re-centralizing coordination by aggregating all agents' plans and resolving conflicts jointly. While effective, this approach relies on the assumption that agents can reliably exchange their plans with one another. This assumption is architecturally load-bearing: without it, the re-centralization step has no data to operate on, and the approach breaks down entirely. Yet reliable communication is far from guaranteed in many real-world deployments. In subterranean environments such as mines or collapsed buildings, radio signals are attenuated or blocked entirely. In adversarial settings, communication channels may be actively jammed. Even in benign environments like dense warehouses, RF interference from metallic infrastructure can degrade signal quality, and as the number of agents scales, the required communication bandwidth grows with it, introducing latency and synchronization failures. Critically, these failure modes are most dangerous precisely in the safety-critical scenarios motivating MAPF in the first place: a robot carrying hazardous materials that loses communication has no fallback coordination mechanism and must act under full uncertainty about its neighbors' intentions.

In this work, we address a more realistic variant of the MAPF problem in which robots cannot communicate with one another. We tackle this challenge along two complementary directions: (1) learning a more conflict-aware policy that

implicitly reduces collisions during decentralized execution, and (2) designing a planning algorithm that reasons explicitly under the uncertainty arising from unknown agent intentions.

II. RELATED WORK

A. MAPF

Multi-agent path finding (MAPF) methods can be broadly grouped into search-based, learning-based, and hybrid approaches. Search-based methods remain the standard choice when correctness and explicit conflict resolution are important, while learning-based methods improve scalability and inference speed. Hybrid methods aim to combine both, using learning for efficiency and planning or shielding for reliability.

1) *Search-based methods*: Search-based MAPF methods explicitly resolve inter-agent conflicts and are the dominant line of work when formal guarantees are required. Conflict-Based Search (CBS) [12] is a representative framework that separates global conflict handling from low-level single-agent planning. More recent methods such as LaCAM [9] improve scalability on large MAPF instances, while Windowed MAPF with Completeness Guarantees [15] shows that even receding-horizon planning can retain completeness.

2) *Learning-based methods*: Learning-based MAPF methods replace expensive online search with reactive policies. Earlier works such as PRIMAL2 [5] and SCRIMP [16] show that decentralized policies can scale well and handle dense environments more efficiently at inference time. More recently, MAPF-GPT [1] and MAPF-GPT-DDG [2] demonstrate that large-scale imitation learning can further improve generalization and solution quality.

3) *Hybrid methods*: Hybrid methods combine learned policies with search-based correction or shielding mechanisms. Recent work shows that augmenting a learned local policy with CS-PIBT [14] can effectively resolve local collisions, and even simple imitation learning can become competitive once combined with such a planner [13].

B. Multi-agent interaction

A key challenge in decentralized multi-agent settings is reasoning about other agents' behaviors under uncertainty. Classical approaches use Hamilton-Jacobi (HJ) reachability to provide formal safety guarantees for multi-vehicle systems, but direct application is intractable at scale due to the curse of dimensionality. Sequential path planning (SPP) [3, 4] addresses this by assigning strict priorities to agents, reducing joint planning complexity from exponential to linear, and extending the framework to handle dynamic disturbances and imperfect knowledge of higher-priority vehicles' states. While these methods offer provable safety, they assume agents can be assigned fixed priorities and that some state information is available — assumptions that break down in fully decentralized, communication-denied settings. More recent work has shifted toward learning-based and game-theoretic approaches to handle the richer uncertainty arising from unknown agent intentions. [17] proposes Sequential Neural Control Barrier Functions (SN-CBFs) that decompose

multi-obstacle interaction patterns into per-obstacle temporal sequences, enabling policies trained on sparse scenarios to generalize to much denser environments. At the game-theoretic level, [11] introduces contingency games, where a robot maintains a distribution over possible intents for other agents and plans a set of contingent trajectories conditioned on different intent hypotheses, interpolating between certainty-equivalent planning and fully robust planning depending on how quickly uncertainty is expected to resolve. [6] extends this further with a closed-loop paradigm that jointly models physical dynamics and the robot's own learning algorithm, using adversarial reinforcement learning to synthesize safe policies that explicitly account for the robot's evolving uncertainty at runtime. Together, these works reveal a spectrum of approaches for multi-agent interaction under uncertainty — from priority-based formal guarantees to adaptive game-theoretic reasoning — each making different assumptions about what information is available and how agents model each other.

III. PROBLEM FORMULATION

We formalize the problem of communication-free decentralized multi-agent path finding as follows.

a) *Environment*.: We are given an undirected graph $G = (V, E)$ representing the environment, where V is the set of traversable vertices and E is the set of edges between adjacent vertices. In grid-based settings, V corresponds to free cells and E connects cells reachable in one step.

b) *Agents*.: A set of N agents $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ operate in G . Each agent a_i has a start position $s_i \in V$ and a goal position $g_i \in V$. All start positions are distinct, and all goal positions are distinct.

c) *Actions and transitions*.: Time is discretized into timesteps. At each timestep t , each agent a_i selects an action from the action set $\{\text{WAIT}, \text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$. The action transitions the agent to an adjacent vertex or keeps it at its current vertex.

d) *Observation model*.: Each agent a_i has a local field-of-view of radius r . At timestep t , agent a_i observes:

- The obstacle structure of G within radius r of its current position,
- The positions of all other agents within radius r ,
- Its own relative goal direction.

We denote the observation of agent a_i at timestep t as o_i^t .

e) *Assumptions*.: We make the following assumptions:

- 1) **Shared policy**. All agents execute the same deterministic policy $\pi : \mathcal{O} \rightarrow \text{Actions}$, where $\mathcal{O}$ is the observation space.
- 2) **Known priority ordering**. A fixed priority ordering $\sigma : \mathcal{A} \rightarrow \{1, 2, \dots, N\}$ is known to all agents prior to execution.
- 3) **Known map**. All agents have access to the full graph G .
- 4) **No communication**. Agents exchange no messages during execution. Each agent selects its action using only its local observation o_i^t , the shared policy π , the priority ordering σ , and the map G .

f) *Safety and completeness constraints.*: A valid execution must satisfy:

- **Vertex collision-free:** $\text{pos}(a_i, t) \neq \text{pos}(a_j, t)$ for all t and all $i \neq j$.
- **Edge collision-free:** For all t and all $i \neq j$, it is not the case that $\text{pos}(a_i, t) = \text{pos}(a_j, t+1)$ and $\text{pos}(a_j, t) = \text{pos}(a_i, t+1)$.
- **Completeness:** For each agent a_i , there exists a finite timestep T_i such that $\text{pos}(a_i, T_i) = g_i$.

g) *Objective.*: Minimize the sum-of-costs $\sum_{i=1}^N T_i$, where T_i is the first timestep at which agent a_i reaches its goal g_i .

h) *Distinction from prior work.*: In standard centralized MAPF, a single planner computes all paths with full knowledge of all agents. In existing decentralized learning-based MAPF (e.g., MAPF-GPT), each agent independently computes an action, but a communication-dependent collision shield (e.g., PIBT) resolves conflicts before execution. In our formulation, the safety and completeness constraints must be satisfied **by the policy and algorithm alone**, without any inter-agent message passing at runtime.

IV. PROPOSED APPROACH

Our approach proceeds in two stages. In Stage 1, we improve the base MAPF-GPT policy by fine-tuning on unsafe states mined via a PIBT safety shield (Section IV-A). This produces a conflict-aware policy π^* that is more robust in dense settings. In Stage 2, we use π^* as the shared policy within an uncertainty-aware planning framework (Section IV-B). Because all agents execute the same policy, agent i can use π^* to simulate the future behavior of higher-priority neighbors and construct a local risk map. The fine-tuned policy thus serves a dual role: it directly improves each agent's action quality, and it provides a more accurate model for predicting other agents' behavior during risk-aware planning.

A. Learning a self-safe policy via unsafe state mining

One potential approach is to improve a pure learning-based MAPF policy by mining unsafe states and using them for supervised fine-tuning. We combine MAPF-GPT with a local CS-PIBT safety shield, and run the hybrid system on dense maps where failures are frequent. Whenever the shield is triggered, we record the corresponding state as an unsafe state and treat the shield's corrective action as expert supervision. We then fine-tune MAPF-GPT on these mined examples so that it can proactively avoid imminent collisions and local deadlocks from local observations. In this way, reactive shielding is converted into proactive policy improvement, reducing the dependence on external correction and improving robustness in dense scenarios.

B. Uncertainty-Aware Planning via Risk Map

Although the learned policy π^* encourages conflict-free behaviour, it cannot guarantee the absence of collisions. We leverage the two assumptions introduced in Section III—fixed priority ordering and full observability within radius r —to

reason explicitly about the future occupancy of higher-priority agents and incorporate this uncertainty into action selection.

1) *Simulating Higher-Priority Agents.*: For agent i with priority σ_i , let $P = \{p_1, p_2, \dots, p_k\} \subset A$ denote the set of higher-priority agents within observation radius r , ordered such that $\sigma_{p_1} > \sigma_{p_2} > \dots > \sigma_{p_k}$. Since all agents share the same policy π^* , agent i can simulate the future behaviour of each agent in P for s fixed timesteps.

The simulation proceeds in priority order. For the highest-priority agent p_1 , since no higher-priority agents exist within the local region, it follows π^* without accounting for others. The policy outputs a probability distribution over actions $\pi^*(a \mid o_{p_1})$. To manage the exponential growth of the trajectory tree, we prune actions with probability below a threshold ϵ and renormalise the remaining distribution:

$$\tilde{\pi}(a \mid o_{p_1}) = \begin{cases} \frac{\pi^*(a \mid o_{p_1})}{\sum_{a': \pi^*(a' \mid o_{p_1}) \geq \epsilon} \pi^*(a' \mid o_{p_1})} & \text{if } \pi^*(a \mid o_{p_1}) \geq \epsilon, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

We then propagate a marginal occupancy probability over the trajectory tree for each simulated agent. Let $r_{ijt}^{(p_\ell)}$ denote the probability that agent p_ℓ occupies cell (i, j) at timestep t . Starting from agent p_ℓ 's known position (i_0, j_0) at $t = 0$, we initialise $r_{i_0 j_0 0}^{(p_\ell)} = 1$ and compute forward recursively:

$$r_{ijt}^{(p_\ell)} = \sum_{(i', j') \in \mathcal{N}(i, j)} r_{i' j' t-1}^{(p_\ell)} \cdot \tilde{\pi}(a_{(i', j') \rightarrow (i, j)} \mid o_{p_\ell}^{t-1}) \quad (2)$$

where $\mathcal{N}(i, j)$ denotes the set of cells that can transition into (i, j) in one step. This yields a marginal occupancy distribution—summing over all trajectory branches—for each cell and timestep.

For subsequent agents $p_2, \dots, p_k$, the simulation proceeds analogously, but each agent avoids cells with high cumulative occupancy risk from higher-priority agents when computing its own trajectory tree.

After simulating all agents in P , we aggregate the per-agent occupancy probabilities into a *cumulative risk map* R_{ijt} by taking the element-wise maximum across agents:

$$R_{ijt} = \max_{p_\ell \in P} r_{ijt}^{(p_\ell)} \quad \forall i, j, t \quad (3)$$

The cumulative risk map R is the quantity used for all downstream planning decisions.

2) *Implicit Risk Propagation.*: The raw risk map R captures direct occupancy probabilities but does not account for the likelihood that a higher-priority agent will *pass through* a cell on its way to its goal. To address this, we propagate risk forward along each agent's likely path using a distance-to-goal map d_{ij} , computed offline via backward Dijkstra from each agent's goal.

For each cell (i, j) with risk R_{ijt} , we propagate to a neighbouring cell (x, y) only if (x, y) is closer to the goal, i.e. $d_{xy} < d_{ij}$:

$$R_{xyt} \leftarrow \max(R_{xyt}, R_{ijt} \cdot \alpha^{d_{ij} - d_{xy}}) \quad (4)$$

where $\alpha \in (0, 1)$ is a decay factor. When $d_{xy} \geq d_{ij}$, the cell is unlikely to lie on the forward path and its risk is left unchanged. This ensures that risk propagates only in the direction of likely agent motion, decaying with distance along the path.

3) *Risk-Aware Action Selection*: Once the risk map R is constructed, agent i at position (i, j) at timestep τ selects its action by integrating the learned policy with the cumulative occupancy risk. For each candidate action a leading to neighbouring cell (x, y) , we define the action cost as:

$$c(a) = -\lambda_1 \log \pi^*(a | o_i) + \lambda_2 \cdot R_{xy, \tau+1} \quad (5)$$

where $-\log \pi^*(a | o_i)$ is the negative log-probability of the action under the learned policy (lower is more preferred), $R_{xy, \tau+1}$ is the cumulative occupancy risk at the destination cell at the next timestep, and $\lambda_1, \lambda_2 > 0$ are weighting hyperparameters that balance policy preference against collision risk. The agent then selects the action with minimum cost:

$$a^* = \arg \min_{a \in \mathcal{A}(i, j)} c(a) \quad (6)$$

where $\mathcal{A}(i, j)$ denotes the set of valid actions from (i, j) , consisting of the four cardinal moves and the wait action.

V. PRELIMINARY RESULTS

A. Benchmarking (Chiawen)

agents	csr_pct	isr_mean	runtime_mean	runtime_p90
8	100.0	1.0	5.312153	8.529402
16	100.0	1.0	16.738215	20.798726
24	99.22	0.9997	29.356402	38.957119
32	96.09	0.9888	46.980182	65.248374
48	84.38	0.9508	98.936414	189.061252
64	52.34	0.8665	114.319089	257.619466

TABLE I

PERFORMANCE STATISTICS UNDER DIFFERENT NUMBERS OF AGENTS.

Table I shows the benchmark results of MAPF-GPT 2M. Here, `num_agents` is the number of agents, `runs` is the number of evaluated instances, `csr_pct` is the case success rate, `isr_mean` is the mean individual success rate, and `runtime_mean/runtime_p90` denote the average and 90th-percentile runtime, respectively. We observe that MAPF-GPT 2M performs well when the number of agents is small, but the success rate drops progressively as the number of agents increases. From visualization, many failed cases are associated with local deadlocks and repeated collisions, where agents are unable to escape congested interactions. This suggests a key limitation of purely learning-based methods in dense MAPF settings and motivates the use of a local safety shield to improve robustness.

B. PIBT-shield (Thomas)

To facilitate the fine-tuning of the MAPF-GPT model via unsafe state mining, we first require a method that can generate safe trajectories for data collection. To achieve this, we implemented a hybrid execution pipeline: at each timestep, the

pre-trained MAPF-GPT model proposes an action for every agent, and a centralised PIBT shield then inspects and, where necessary, overrides these proposals to resolve and proactively avoid collisions. States in which the shield intervenes are recorded as unsafe states, and the shield’s corrective actions serve as expert labels for subsequent fine-tuning.

In a preliminary 48-agent configuration, this hybrid system—with the PIBT shield active at runtime—increased the individual agent success rate from 0.35 (MAPF-GPT alone) to 1.00. It is important to note that this result reflects the *data-collection configuration*, not the performance of a fine-tuned policy operating without the shield. Evaluating whether the fine-tuned policy π^* can achieve comparable safety improvements in a fully decentralised, shield-free setting is the primary objective of our next phase of experiments (see Section VI).

VI. NEXT STEPS

A. Milestones and Semester Work Plan

Our immediate next step focuses on data collection for model fine-tuning. We aim to complete this data collection phase and commence the fine-tuning process by the end of March. In parallel, we will begin implementing the uncertainty-aware planning module. Our goal is to finalize this implementation by the end of the first week in April, at which point we will integrate the fine-tuned model directly into the planning framework.

B. Risks & Mitigation Plan

We have identified two primary risks associated with our proposed research directions:

1) Risk 1: Suboptimal Fine-Tuned Model Performance:

There is a possibility that the model’s performance may not meet our expectations even after the fine-tuning process.

Mitigation: To address this, we plan to focus on curating higher-quality training data. Rather than indiscriminately logging all plans generated by the PIBT shield, we will rigorously define what constitutes an “unsafe state” to ensure the collected data is highly targeted and relevant to boundary-case failures.

2) Risk 2 - *Computational Inefficiency in Uncertainty-Aware Planning*: Executing forward simulations for lower-priority agents presents a significant potential computational bottleneck. Because the algorithm must simulate possible trajectories for multiple other agents, the computational complexity risks growing exponentially.

Mitigation: We will initially constrain the simulation by utilizing a short time horizon and a high dropout threshold for low-probability actions. If the algorithm achieves acceptable performance and runtime under these constraints, we will incrementally extend the simulation horizon and allow for wider trajectory branching.

REFERENCES

- [1] Anton Andreychuk, Konstantin Yakovlev, Aleksandr Panov, and Alexey Skrynnik. Mapf-gpt: Imitation learning for multi-agent pathfinding at scale, 2025. URL <https://arxiv.org/abs/2409.00134>.

- [2] Anton Andreychuk, Konstantin Yakovlev, Aleksandr Panov, and Alexey Skrynnik. Advancing learnable multi-agent pathfinding solvers with active fine-tuning, 2025. URL <https://arxiv.org/abs/2506.23793>.
- [3] Somil Bansal, Mo Chen, Jaime F Fisac, and Claire J Tomlin. Safe sequential path planning under disturbances and imperfect information. *arXiv preprint arXiv:1603.05208*, 2016.
- [4] Mo Chen, Somil Bansal, Ken Tanabe, and Claire J Tomlin. Provably safe and robust drone routing via sequential path planning: A case study in san francisco and the bay area. *arXiv preprint arXiv:1705.04585*, 2017.
- [5] Mehul Damani, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. Primal₂: Pathfinding via reinforcement and imitation multi-agent learning - lifelong. *IEEE Robotics and Automation Letters*, 6(2):2666–2673, April 2021. ISSN 2377-3774. doi: 10.1109/LRA.2021.3062803. URL <http://dx.doi.org/10.1109/LRA.2021.3062803>.
- [6] Haimin Hu, Zixu Zhang, Kensuke Nakamura, Andrea Bajcsy, and Jaime F Fisac. Deception game: Closing the safety-learning loop in interactive robot autonomy. In *Conference on Robot Learning (CoRL)*, 2023.
- [7] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. Eecbs: A bounded-suboptimal search for multi-agent path finding, 2021. URL <https://arxiv.org/abs/2010.01367>.
- [8] Qingbiao Li, Weizhe Lin, Zhe Liu, and Amanda Prorok. Message-aware graph attention networks for large-scale multi-robot path planning, 2021. URL <https://arxiv.org/abs/2011.13219>.
- [9] Keisuke Okumura. Lacam: Search-based algorithm for quick multi-agent pathfinding, 2022. URL <https://arxiv.org/abs/2211.13432>.
- [10] Keisuke Okumura, Manao Machida, Xavier Défago, and Yasumasa Tamura. Priority inheritance with backtracking for iterative multi-agent path finding. *Artificial Intelligence*, 310:103752, September 2022. ISSN 0004-3702. doi: 10.1016/j.artint.2022.103752. URL <http://dx.doi.org/10.1016/j.artint.2022.103752>.
- [11] Lasse Peters, Andrea Bajcsy, Chih-Yuan Chiu, David Fridovich-Keil, Forrest Laine, Laura Ferranti, and Javier Alonso-Mora. Contingency games for multi-agent interaction. *IEEE Robotics and Automation Letters*, 2024.
- [12] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015. ISSN 0004-3702. doi: <https://doi.org/10.1016/j.artint.2014.11.006>. URL <https://www.sciencedirect.com/science/article/pii/S0004370214001386>.
- [13] Rishi Veerapaneni, Arthur Jakobsson, Kevin Ren, Samuel Kim, Jiaoyang Li, and Maxim Likhachev. Work smarter not harder: Simple imitation learning with cs-pibt outperforms large scale imitation learning for mapf, 2024. URL <https://arxiv.org/abs/2409.14491>.
- [14] Rishi Veerapaneni, Qian Wang, Kevin Ren, Arthur Jakobsson, Jiaoyang Li, and Maxim Likhachev. Improving learnt local mapf policies with heuristic search, 2024. URL <https://arxiv.org/abs/2403.20300>.
- [15] Rishi Veerapaneni, Muhammad Suhail Saleem, Jiaoyang Li, and Maxim Likhachev. Windowed mapf with completeness guarantees. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(22):23323–23332, April 2025. ISSN 2159-5399. doi: 10.1609/aaai.v39i22.34499. URL <http://dx.doi.org/10.1609/aaai.v39i22.34499>.
- [16] Yutong Wang, Bairan Xiang, Shinan Huang, and Guillaume Sartoretti. Scrimp: Scalable communication for reinforcement- and imitation-learning-based multi-agent pathfinding, 2023. URL <https://arxiv.org/abs/2303.00605>.
- [17] Hongzhan Yu, Chiaki Hirayama, Chenning Yu, Sylvia Herbert, and Sicun Gao. Sequential neural barriers for scalable dynamic obstacle avoidance. 2023.